此为临时预览链接，将在短期内失效

05 本体的“星际导航”：如何从混沌业务中识别和定义领域本体?

陈乔 通往AI之桥 2026年10月27日 10:00

从业务文档、SQL、API 和专家经验中，找到机器与人都能理解的业务世界模型通往AI之桥 | 本体论在 AI 工程中的实践系列 · 05

我们已经知道，本体（Ontology）可以帮助 AI 系统统一业务概念、关系与规则。

但真正进入企业项目，另一个更难的问题马上就会出现：**本体从哪里来?**

业务部门有流程文档，开发团队有数据库和 API，运维团队有日志，架构师有设计图，资深员工则把大量经验放在脑子里。不同来源使用不同的词，同一个词又可能有不同含义；有些规则写在制度里，有些藏在代码中，还有一些只有出了故障才会被大家想起来。

如果一开始就要求大家“画一张完整的本体图”，项目很容易变成术语争论；如果把所有文档丢给大模型，让它直接生成本体，又可能得到一张看起来很完整、却与真实业务不一致的图。

本体工程真正需要的，不是一次“自动画图”，而是一套从业务证据出发、持续澄清和验证语义的过程。

> **本体不是从数据库表里直接抄出来的，也不是让大模型凭空想出来的。它应当从业务问题出发，在多种证据中发现候选概念，再通过业务验证形成可维护的语义模型。**



【图 1 | 穿越混沌业务数据的“语义导航”】

一、不要先问“有哪些表”，先问“我们要解决什么问题”

此为临时预览链接，将在短期内失效

页面原样呈现一遍。

数据库描述的是某个系统如何存储和处理数据；本体试图描述的是某个业务领域中，哪些概念具有独立含义、彼此如何关联，以及系统需要遵循哪些约束。两者相关，但不是同一件事。

例如，某个软件工厂系统里可能有 artifact、build_job、test_result、release_record 等表。单纯按表建模，很容易得到一份数据库结构图，却未必能回答这些真正重要的问题：

- 哪个产品版本包含了这个组件？
- 某次构建使用了哪些源代码提交和依赖版本？
- 哪些测试结果可以作为发布审批的依据？
- 一个高危漏洞会影响哪些产品版本和客户环境？
- 如果客户要求升级，当前组件组合是否满足兼容性要求？

这些问题决定了本体需要表达什么，而不是反过来让数据库里有什么就建模什么。

因此，开始前应先写下一份简短的“本体项目章程”：

要回答的问题	示例
业务范围是什么？	软件产品的构建、测试、发布和客户部署
谁会使用？	产品经理、开发 Agent、安全 Agent、发布负责人和运维 Agent
最重要的决策是什么？	判断某个版本能否发布，以及漏洞影响哪些部署环境
哪些问题必须能被回答？	版本追溯、依赖分析、发布条件检查、影响范围分析
本期明确不做什么？	不建模企业所有业务，也不重建完整的 CMDB 或资产管理系统

这个步骤看起来很朴素，却会决定后面本体的规模和复杂度。

经典的《Ontology Development 101》同样建议先确定领域范围、用途、使用者和本体应回答的问题，并强调本体建模没有唯一正确的方式，应该迭代开发、持续修订。[1]

二、用“能力问题”反推本体：先定义什么叫建模成功

在本体工程中，有一个很实用的办法：先写出系统必须能够回答的问题，再据此判断需要哪些概念和关系。这类问题通常称为**能力问题（Competency Questions）**。

以软件工厂为例，可以先写出以下问题：

此为临时预览链接，将在短期内失效

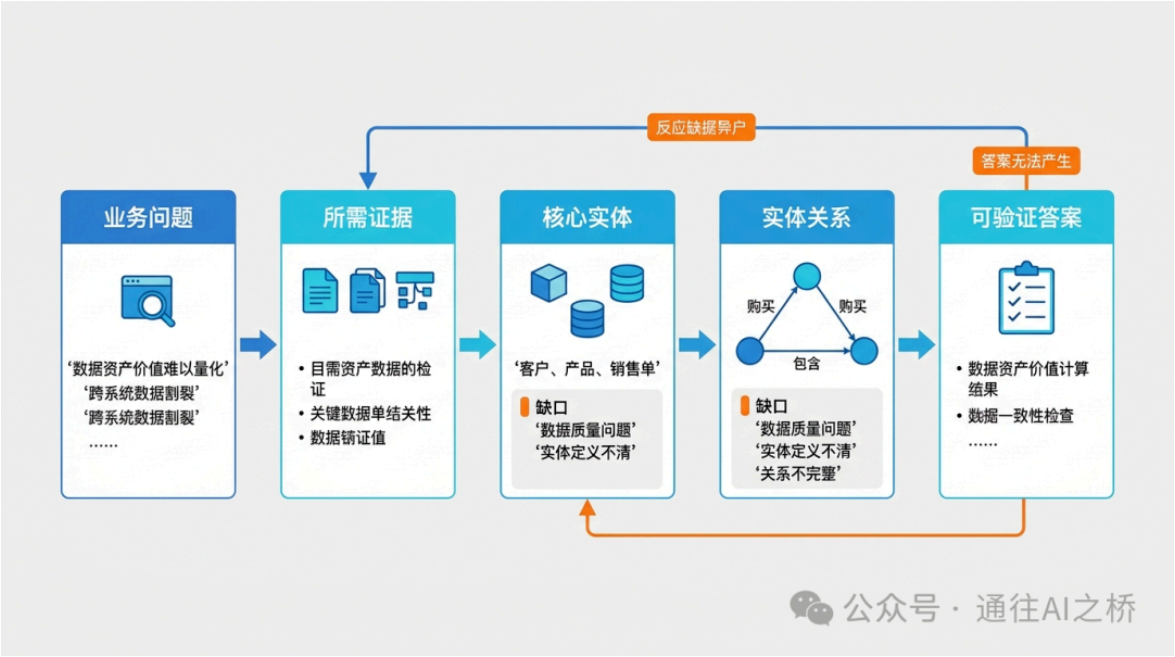
3. 一个候选版本是否满足发布条件？缺少什么证据?
4. 某个客户环境当前部署了哪些产品版本和组件版本?
5. 如果一个基础组件升级，哪些产品和部署环境需要回归测试?

这些问题不是为了展示模型有多聪明，而是给本体划定边界。

例如，要回答“哪些客户环境受到漏洞影响”，系统至少需要知道：漏洞影响哪些组件版本、产品版本包含哪些组件版本、客户环境部署了哪些产品版本。假如本体中只有“产品”和“客户”两个概念，却没有版本和部署关系，那么这个问题就无法可靠回答。

能力问题也能够暴露过度建模。假如当前只需要判断版本发布是否满足质量要求，就没有必要一开始就描述整个企业的所有组织关系、采购合同和财务科目。

一个好用的本体，不是拥有最多的类和关系，而是能够以清晰、可验证的方式回答目标问题。



【图 2｜能力问题如何反推本体】

三、从五类业务材料中发现候选概念

真实企业里的业务知识通常散落在不同材料中。不同来源能提供的证据也不同，因此最好不要只依赖一种材料。

1. 业务文档：发现业务语言和显式规则

制度、需求文档、操作手册、合同模板、SOP 和产品说明，通常包含大量业务名词、角色、条件和流程描述。

此为临时预览链接，将在短期内失效

> 生产发布前，必须完成回归测试；存在未豁免的严重漏洞时，不得进入生产环境。

这里至少藏着一些候选概念：生产发布、回归测试、严重漏洞、豁免、生产环境。更重要的是，它包含了规则和约束：发布需要哪些前置证据，哪些异常需要被阻断，什么情况下允许豁免。

但文档里的词不能直接自动升级为本体类。某些词可能只是临时项目用语；某些要求可能已经过期；某些句子可能只是建议，而不是强制规则。提取出来的只是候选项，后续还要确认。

2. 数据库 SQL：发现事实结构与系统口径

SQL、表结构、视图、约束和数据字典，能帮助我们理解现有系统实际存储什么、使用什么标识，以及数据之间有哪些显式连接。

例如：

- `product_id`
  可能是产品的标识；
- `version_id`
  可能指向一个组件版本或产品版本；
- 外键关系可能表达构建记录与制品之间的关联；
- 状态字段可能包含 `READY`、`FAILED`、`RELEASED` 等值。

但不能直接假定字段名就是业务定义。version 在不同表中可能分别表示产品版本、API 版本、配置版本或数据库 Schema 版本。数据表也可能保存历史技术债务或不完整约束。

因此，SQL 更适合作为“现有事实结构”的证据，而不是领域语义的唯一权威。

3. API：发现系统对外提供的业务能力

API 文档、OpenAPI Schema、请求和响应示例，可以帮助识别系统支持哪些操作、输入输出是什么，以及哪些实体通过接口相互关联。

例如：

- `POST /releases`
  ：创建发布申请；
- `GET /components/{id}/versions`
  ：获取组件版本；

此为临时预览链接，将在短期内失效

API 名称并不一定能准确描述业务含义。一个叫 release 的接口可能创建的是发布申请，也可能执行实际生产发布，还可能只是把构建产物推送到制品仓库。必须检查接口的副作用、状态变化、权限和调用约束，才能知道它究竟对应什么业务动作。

4. 事件日志：发现真实过程、状态变化和异常路径

事件日志经常比流程文档更接近系统实际运行情况。它能帮助识别正常流程以外的状态转换、失败原因、重试模式和隐含依赖。

例如：

BuildCreated → TestFailed → BuildRetried → TestPassed → ReleaseApproved

这条事件序列提示我们：构建、测试、重试和发布审批可能是不同类型的事件；测试失败并不一定导致流程终止；发布批准也不等于已经完成部署。

不过，日志是一种观察结果，不一定能解释背后的业务意义。status=done 可能只表示一个任务结束，不代表业务目标已经完成。要结合事件定义、系统实现和领域专家的解释，才能建立可靠模型。

5. 专家访谈：发现文档中没有写出来的知识

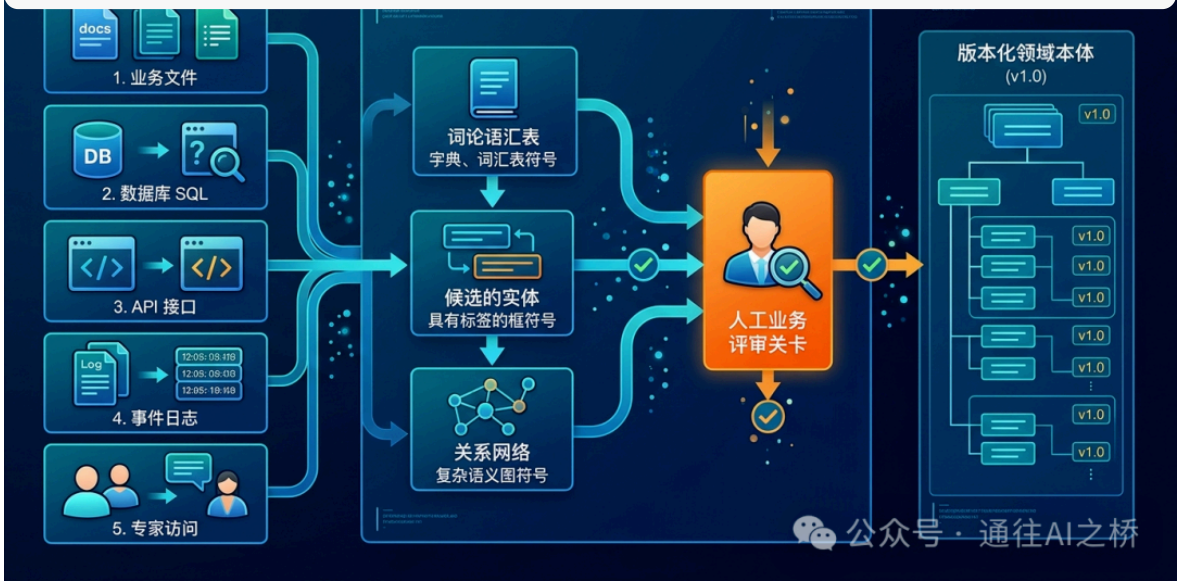
领域专家往往知道一系列难以从文档中直接读出的规则，例如：

- 哪些告警虽然严重，但在特定条件下可以延期处理？
- 哪种“完成”只是技术任务结束，哪种才是客户验收完成？
- 哪些字段看似相同，实际上有不同的结算口径？
- 某个异常为什么需要先回滚，而不是直接重试？

访谈时，不要只问“这个业务有哪些概念”。更有效的方法是让专家讲一次具体任务：输入是什么、做了哪些判断、检查哪些证据、什么时候需要升级处理、哪些情况属于例外。

然后用异常案例反问：如果数据缺失怎么办？如果两个系统结论冲突怎么办？如果测试通过但依赖版本不兼容怎么办？这些问题通常比抽象讨论更容易揭示实际规则。

此为临时预览链接，将在短期内失效



【图 3｜五类证据汇聚成候选领域模型】

四、识别四类语义陷阱：同名异义、异名同义、概念重叠和隐含规则

候选概念收集完成后，真正困难的工作才刚刚开始：**同一个词在不同材料中可能不是同一个概念，不同的词又可能指向同一个概念。**

陷阱一：同名异义

例如，“版本”可能指：

- 产品版本（Product Version）；
- 组件版本（Component Version）；
- API 版本（API Version）；
- 配置版本（Configuration Version）。

它们有时共享同一个字段名，却具有不同的生命周期和依赖关系。不能因为名称相同，就建立一个含义模糊的万能 Version 类。

一个实用做法是先保留各来源中的原始术语，再记录它的上下文、定义、来源系统、责任团队和代表性实例。确认含义后，再决定是独立概念、上位概念，还是同一概念的不同表现形式。

陷阱二：异名同义

例如，“组件包”“制品”“构建产物”在不同团队中可能指向同一类对象，也可能指向不同生命周期阶段的对象。

不能仅凭词面相似就把它们合并。要比较它们的：

此为临时预览链接，将在短期内失效

- 属性、关系和约束是否相同；
- 是否会在同一个问题中产生不同答案。

如果它们确实指向同一概念，可以将一个名称作为规范术语，其余作为别名或来源术语保存；如果语义不同，就应保持区分，并定义必要的映射关系。

陷阱三：概念重叠或边界不清

比如，“发布”“部署”“上线”在日常对话中可能混用，但在生产系统中通常需要区别：

- 发布（Release）：某个版本被批准并形成可交付的软件发布项；
- 部署（Deployment）：把某个制品安装或应用到指定环境；
- 上线（Go-live）：业务开始正式使用，可能还涉及切换、验收和运营准备。

这些定义只是一个可选的工程约定，具体企业应根据自身流程确定。关键不是全行业必须采用同一套词，而是组织内部要明确区分，并让系统、指标和 Agent 遵循同一约定。

陷阱四：隐含业务规则

有些规则不会以“规则”的形式出现在文档里，而是隐藏在流程和例外处理中。例如：

- 通过测试的版本仍可能因许可证问题不能发布；
- 生产部署必须使用经过签名的制品；
- 高危漏洞可以申请豁免，但必须有责任人、期限和审批记录；
- 测试通过的是某个构建，不应自动被解释为所有相同版本的构建都通过。

这些规则通常通过“正常案例 + 反例 + 例外案例”来发现。访谈、事件日志和现有自动化流水线都可以提供线索，最终要由领域负责人确定其业务效力。

五、从术语到本体：一套可重复执行的七步法

下面给出一套适合企业 AI 项目的轻量流程。它不是唯一方法，但可以作为从混沌材料走向可评审本体的起点。

第一步：限定范围与使用场景

先定义本体服务哪个业务领域、由谁使用、用于回答什么问题，以及明确暂时不建模什么。范围太大，会让建模工作失去边界；范围太小，则可能无法支持目标问题。

第二步：收集多源证据

此为临时预览链接，将在短期内失效

谈记录。为每条证据保留来源、版本、日期和可信度，不要把推断与原始事实混为谈。

第三步：抽取候选术语和定义

提取名词、动词、状态词、条件和规则。名词可能提示实体或属性，动词可能提示关系或动作，条件句可能提示约束。但它们只是线索，不是自动建模规则。

第四步：消歧并建立术语表

为每个候选概念记录：规范名称、业务定义、别名、来源、责任人、示例和反例。对同名异义、异名同义和概念边界不清的项，明确记录待确认的问题，不要急于合并。

第五步：用能力问题反推类、属性和关系

从要回答的问题中识别必须存在的概念和关系。例如，要回答漏洞影响范围，就需要漏洞、组件、版本、产品和部署环境等概念，以及它们之间的影响、依赖、包含和部署关系。

第六步：加入关键约束并映射数据源

为概念补充必要的属性、关系方向、基数、状态和约束，再将它们映射到数据库字段、API、事件和文档。不要要求本体复制所有物理字段；只映射业务问题真正需要的内容。

第七步：用真实问题和反例验证，持续迭代

让业务专家和技术团队共同检查：能力问题能否被回答？答案是否符合业务事实？异常情况会不会被错误处理？数据源变化后，本体是否仍然成立？将验证结果和修订记录一起纳入版本管理。

经典的 Ontology Development 101 强调，领域本体开发是迭代过程，应通过实际应用和领域专家反馈不断调试和修订，而不是期望第一次设计就正确。[1]

六、用软件工厂案例走一遍：第一版本体应该长什么样？

假设我们当前只关心“漏洞影响范围与发布决策”，可以从一组小型概念开始：

概念	含义	关键关系或属性
Requirement（需求）	需要被实现或验证的业务/工程要求	由某产品版本实现

此为临时预览链接，将在短期内失效

SoftwareProduct（软件产品）	可交付的软件产品	包含产品版本
Component（组件）	可复用的软件组件	有多个组件版本
ComponentVersion（组件版本）	某个可识别的组件版本	依赖其他组件版本；可能受漏洞影响
Build（构建）	某次构建执行及其结果	使用代码提交和依赖；产生制品
TestRun（测试运行）	某次测试执行及结果	验证某个构建或制品
Finding（发现项）	漏洞、质量问题或合规发现	影响特定版本或制品
Release（发布项）	经流程管理的交付版本	包含制品；具有审批和发布状态
Deployment（部署）	某制品在目标环境中的部署记录	部署到客户或运行环境

它们之间的关系可以先用自然语言表达：

- 产品版本包含组件版本；
- 构建使用指定的源代码提交和依赖版本；
- 构建产生制品；
- 测试运行验证某个构建或制品；
- 漏洞发现项影响一个或多个组件版本；
- 发布项包含经过审核的制品；
- 部署记录说明某个制品在哪个环境运行。

不要急着把它们全部写成 OWL 或图数据库 Schema。第一轮先让业务负责人确认概念边界与关系，再挑选少量关键问题做验证。例如：

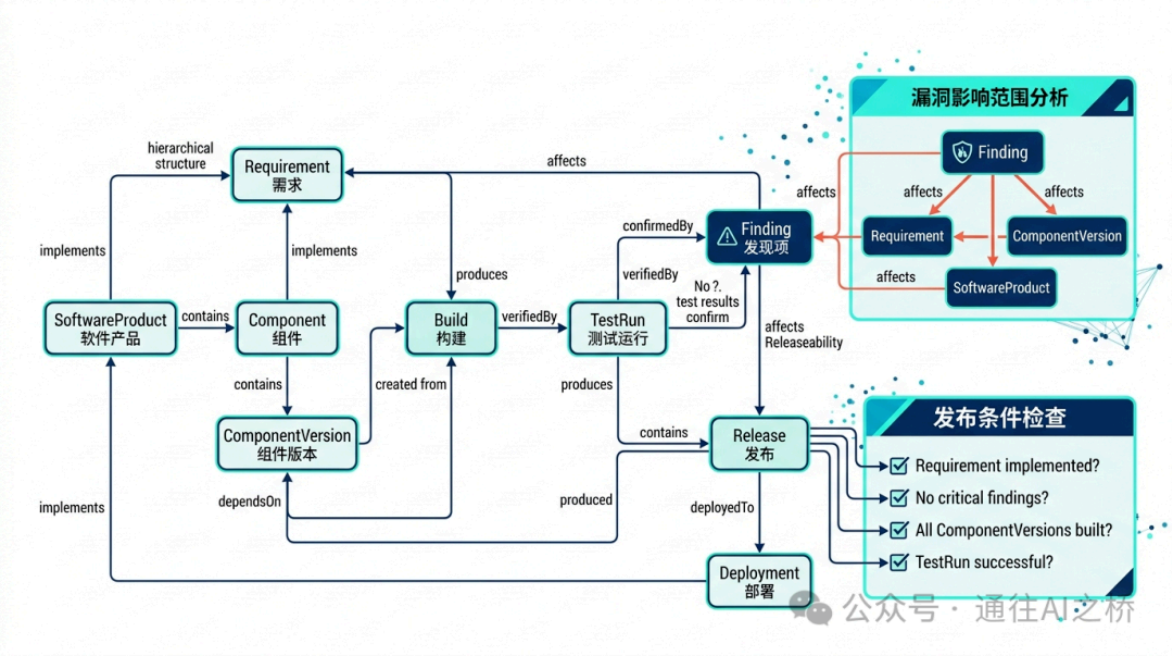
问题 A：漏洞 CVE-X 影响哪些客户环境？需要沿着“漏洞 → 组件版本 → 产品版本/制品 → 部署环境”追踪，并确保版本信息准确。

问题 B：某个版本能否发布？需要检查构建、测试、漏洞发现、豁免和审批记录。注意：本体可以表达相关对象和约束，实际阻止发布仍应由 CI/CD、策略引擎和审批工作流执行。

问题 C：某次升级需要回归测试哪些产品？需要知道组件版本的依赖关系、产品包含关系和受影响范围。

此为临时预览链接，将在短期内失效

收敛本体的过程。



【图 4｜软件工厂领域本体草图】

七、大模型可以参与多少？让它做建模助手，不让它做最终裁判

大模型能够显著加快本体工程中的一些工作：

- 从文档、SQL、API 定义和日志中提取候选术语；
- 聚合同义词，标记可能存在歧义的术语；
- 根据业务问题建议实体、属性和关系；
- 生成初版术语表、本体草图和能力问题清单；
- 为候选模型生成示例数据、反例和测试问题；
- 对比两个本体版本的变化，并提示可能受影响的查询或规则。

但大模型也可能把“经常一起出现的词”误认为同一个概念，把代码中的历史实现误认为正确的业务规则，或编造没有证据支持的关系。因此，建议为每个抽取结果保留来源片段、推断理由、置信度和待确认状态。

一个可行的协作流程是：

1. LLM 提取候选项，并附上原始证据；
2. 本体工程师对概念合并、边界和关系作初步判断；
3. 领域专家确认定义、例外和业务效力；
4. 工程工具检查结构约束，并用能力问题验证结果；
5. 通过评审的变更进入版本库，保留差异和审批记录。

此为临时预览链接，将在短期内失效

量，而模型在早期概念化阶段表现相对更好，在后续形式化阶段仍需要更强的人工验证。[2] 这项研究不能直接代表所有行业，但支持一个值得采用的工程原则：**让 AI 加速候选发现和初稿生成，让领域专家与验证机制负责确认语义。**

八、怎样判断第一版本体是否合格?

不要用“画了多少节点”或“抽取了多少术语”作为主要成绩。更值得关注的是：

- **覆盖度：** 目标能力问题需要的概念和关系是否齐全?
- **一致性：** 同名异义和异名同义是否得到处理？重要约束是否相互冲突?
- **可追溯性：** 每个关键定义和关系能否找到来源或责任人?
- **可验证性：** 是否有典型问题、反例和测试数据验证本体能否支持预期任务?
- **可维护性：** 概念变更能否被评审、版本化，并评估对查询、Agent 和系统映射的影响?
- **实际收益：** 是否减少语义误解、人工返工、错误查询或不必要的工具调用?

本体工程不是词汇整理比赛，而是让系统在目标问题上形成更稳定的理解能力。若增加本体后没有改善任何关键任务，反而增加大量维护成本，就应该缩小范围或重新审视设计。

九、结语：本体工程是一场持续校准，而不是一次性建图

从混沌的业务材料中发现本体，像一次“星际导航”：业务文档、SQL、API、事件日志和专家经验是分散的观测信号；能力问题决定我们要抵达哪里；术语消歧和关系建模帮助我们构建坐标系；业务评审、数据映射和测试则不断校准方向。

这条路径不一定需要一开始就使用最复杂的本体语言，也不要求把企业每一个细节都形式化。更现实的做法是，先从一个高价值业务问题开始，建立最小可用的领域模型，用真实数据和异常案例验证，再逐步扩展。

当本体能够回答实际问题，来源可追溯，重要约束可验证，并且能被不同系统和 Agent 重复使用时，它才从一张概念图变成了真正的工程资产。

> **先确定要回答的问题，再发现需要的概念；先用证据提出模型，再由业务验证语义。不要让本体从混沌中凭空长出来。**

系列预告

下一篇将继续讨论本体如何面对业务变化：

第 06 篇 | 领域本体建模实战：范围界定、能力问题与建模原则

此为临时预览链接，将在短期内失效

– 使用 Competency Questions（能力问题）检验模型是否覆盖真实需求。

– 讨论粒度、命名、复用、模块化和概念层级。

– 识别过度建模、循环定义、把流程写成实体、把数据表直接当作领域模型等常见反模式。

参考资料

1. Natalya F. Noy、Deborah L. McGuinness，**Ontology Development 101: A Guide to Creating Your First Ontology**，Stanford University：https://protege.stanford.edu/publications/ontology_development/ontology101–noy–mcguinness.html
2. **Enhancing Ontology Engineering with Large Language Models: A Stage–Wise Human–in–the–Loop Study**，2026：https://journals.sagepub.com/doi/full/10.3233/SHTI260304

收录于 本体论在AI工程中的实践
作者提示: 内容由AI生成